

绪论

时间复杂度：顽性（立方及以上）/易性算法

空间复杂度：只计辅助空间

线性结构

线性表

顺序结构：内存中连续空间

链式结构：单链表、单循环链表、双链表、双循环链表

栈

顺序栈、链式栈

应用：括号匹配、表达式计算（中缀转后缀）

队列

顺序循环队列（front 队首，rear 队尾的下一个，不需要移动元素）

- 判空：`rear == front`
- 判满：`(rear+1)%n==front`

链式队列

优先队列

树

节点的度：儿子数量

树的度： $\max\{\text{节点的度}\}$

二叉树

特殊形态：

- 满二叉树： k 层共 $2^k - 1$ 个元素
- 完全二叉树： k 层，前 $k - 1$ 层是满的，最后一层所有节点都靠左

性质：

- n 个节点的完全二叉树高度 $k = \lceil \log_2 n \rceil + 1$

- n 个节点的完全二叉树有 $\lceil n/2 \rceil$ 个叶子节点

遍历：

- 前序遍历：中左右
- 中序遍历：左中右
- 后序遍历：左右中
- 层次遍历

遍历序列确定二叉树：

- 完全二叉树：已知层次遍历
- 满二叉树：已知前序、中序、后序之一
- 一般二叉树：前序+中序、中序+后序

线索二叉树：

- 如果左/右孩子为空，则分别用来存某个遍历序列该节点的前驱/后继
- 性质：（中序线索树上）可脱离栈实现中序遍历

森林/树的二叉树存储法：

- 左孩子右兄弟表示法
- 性质：
 - 树：前序遍历不变，后序遍历即二叉树的中序遍历
 - 森林：先序遍历、中序遍历不变

优先级队列

堆，进出队 $O(\log n)$ ，建堆 $O(n)$

在完全二叉树中，

- 任意一个节点的值比其左右子节点都大，称大顶堆
- 都小，称小顶堆

操作：

- 进队：接入树成为叶子节点，不停与父节点比较交换
- 出队：摘取堆顶；将尾部元素放到堆顶，不停与子节点比较交换
- 出队交换方式：找父、兄弟、自己中最小的点放到父上（小根堆）

哈夫曼树

哈夫曼树是一棵最优二叉树（with min Weighted Path Length）

步骤：不停选取森林中节点值最小的两个节点，合并为一棵树，根节点权重为子节点之和

应用：前缀编码（任一个编码都不为其他编码的前缀）

并查集

集合的一个划分中的每个子集称为不相交集，又称并查集

操作：

- 合并（合并两个不相交集）
- 查找（定位元素所属不相交集）

树形存储：

- 每个不相交集为一棵树，根节点即集的编号
- 查找：向上找根
- 合并：将一个根视作另一个根的子节点
- 路径压缩：到根上的所有节点都挪动为根节点的子节点



$$G = (V, E)$$

有向图、无向图

出度、入度

特殊的图：

- 完全图：两两相连
- 连通图：无向图，任两个点都连通 ($E \geq V - 1$)；连通分量
- 强连通图：有向图，任两个点都连通；强连通分量
- 连通图的生成树：其极小连通子图，不唯一

存储：

- 邻接矩阵、邻接表
- 逆邻接表、多重邻接表（同一条边只存储一次）、十字链表

遍历：

- DFS
- BFS

连通性：

- 有向图、无向图的连通性
- 欧拉定理
 - 一条路径经过每条边恰好一次，则称欧拉路径；具有欧拉回路的图称欧拉图
 - 度为奇数的顶点为 0 个：欧拉回路；0 个或 2 个：欧拉路径

AOV（拓扑排序）

- 不断：将入度为 0 的点全部入队列，将队列内点射入的点入度减一

AOE（关键路径）

- 起点到终点的最长路径
- 求顶点最早时间、最晚时间；进而求边最早开始时间、最晚开始时间；最早、最晚开始时间一致的边为关键活动；关键活动构成关键路径

最小代价生成树

Prim

- 不断：找到距离最小的点，更新该点的所有邻居的距离，然后将该点加入树

Kruskal

- 将边按边权从小到大排序
- 对于每条边，如果加入后会形成环，则放弃；否则加上这条边

最短路径

Dijkstra（单源）

- 维护各顶点到源点的最短距离
- 每次从有距离的距离最小的出发，维护完成后把这个点放入 `vst`
- 边权不可为负
- $O(n^2)$

Floyd（所有节点）

- 逐个加入中间节点，对任意两个节点，更新最短距离
- 边权可以为负，但不能有负环
- $O(n^3)$

查找

静态查找

顺序查找 $O(n)$

折半（二分）查找

- 有序， $O(\log n)$
- `while(l<=r), m=(l+r)/2, r=m-1 if x<a[m] else l=m+1`

插值查找

- 有序，已知分布
- 预估位置

分块查找

- 块间有序

二叉查找树

左子树所有节点都小，右子树所有节点都大

操作：

- 查找：自顶向下
- 插入：自顶向下找位置
- 删除：替换为左子树上最大节点或右子树上最小节点

应用：第 k 小元素

- 法一：维护每个节点子树节点数量，自顶向下
- 法二：中序遍历

平衡二叉查找树 (AVL)

平衡因子：左子树高度-右子树高度，平衡二叉树所有节点 $|\text{平衡因子}| \leq 1$

操作：

- 插入
 - 向上传递，直到一个节点的平衡因子变为 0
 - 遇到冲突（不满足AVL定义）时，进行调整
 - 四种冲突类型：LL型（ T 的左孩子的左子树太长，其他同理），LR型，RL型，RR型
 - LL型，右旋 T ；RR型，左旋 T
 - LR型，左旋 T 的左孩子，使其变为LL型；RL型同理
 - 左/右旋 T ：用 T 的右/左儿子替代 T 的位置， T 作为这个儿子的左/右儿子；这个儿子的原左/右儿子移动到 T 的右/左儿子
- 删除
 - 向上传递，直到一个节点的平衡因子由 0 变为非 0
 - 遇到冲突时进行调整

参考：[平衡的调整过程](#)

红黑树

左右子树高度差最大可达一倍

定义：

- 根节点为黑色
- 任一节点，到达子树中任一叶子节点的路径上，黑节点个数相等（叶子节点也算进去）
- 不得有连续的红节点

性质：

- 如忽略红节点，则红黑树是满二叉树

- 高度差为什么是一倍：最长路径黑红相间，最短路径全黑

外部查找

B树

M 阶 B 树满足：

- 根节点儿子数量 $\in [2, M]$
- 除根节点、叶子节点，儿子数量 $\in [\lceil M/2 \rceil, M]$
- 每个节点存储：若干（设为 k ）个值，每个儿子表示这 k 个值分成的 $k + 1$ 个区间中的一个
- 叶子节点：为空，相当于查找失败

操作：

- 插入
 - 查找应插入的节点位置
 - 如果未满，直接插入
 - 如果已满，分裂该节点；如果父节点也满，则继续向上分裂
- 删除
 - 用右侧孩子的最小值替代，或左侧孩子的最大值
 - 如果达下限，可以从兄弟节点拿一个值；如果都到下限，就合并

B+树

与 B 树不同点：

- 叶子节点存储了所有关键字信息
- 每个叶子关键字数量 $\in [\lceil L/2 \rceil, L]$
- 每个叶子节点有指针指向其右侧叶子节点

便于按序遍历

每个节点可以设计为一个磁盘块大小

哈希查找

负载系数：存储的元素与哈希表大小的比值

散列表大小：经验表明素数合适

冲突解决：

- 线性探测法：找下一个空的位置
- 二次探测法： $\pm i^2$ ，第一次 $+1^2$ ，第二次 -1^2 ，...
- 链地址法：散列表只存储单链表的首节点，元素存在单链表中

排序

稳定性：关键值相同的数据，在排序后先后顺序是否发生变化

注：只要不是相邻元素两两交换，**一般**都不稳定

内排序

冒泡排序

- $O(n^2)$ ；正序时可达 $O(n)$
- 早停：全对则停止

插入排序

- $O(n^2)$ ；正序时可达 $O(n)$
- 从后往前找位置

选择排序（不稳定）

- $i \sim n - 1$ 范围内的最小值换到 i 下标位置
- $O(n^2)$

希尔排序（不稳定）

- step：从 $n/2$ 开始，每次减半
- 对步长为 step 的子序列进行插入排序

归并排序

- 从 n 个长度为 1 的有序序列开始
- 将两个有序序列合并为一个
- 合并方法：双指针
- $O(n \log n)$

快速排序（不稳定）

- 选择一个元素为标杆，小的放前面，大的放后面，再分别进行同样的排序
- 找第一个大于标杆，与标杆交换；换方向，找最后一个小于标杆，与标杆交换；不停进行
- 数据基本无序， $O(n \log n)$ ；数据基本有序，标杆在头或尾， $O(n^2)$

堆排序（不稳定）

- （见优先级队列）
- 适合：只需要前 k 个
- 建堆：形式上 $O(n \log n)$ ，实际可达 $O(n)$ ；摘顶 $O(n \log n)$

基数排序

- LSD，按个位数分配再合并，然后按十位数分配再合并
- 又称：口袋排序法
- 适合：正整数
- $O(mn)$ ，其中 m 为最大值的位数

外排序

有 n 个元素，内存一次性只能读入 p 个

主要利用：归并排序

- 预处理：分批读入，进行内排序，形成 m 个已排序片段（每个片段包含 $\leq p$ 个元素）
- 归并： k 路归并，归并次数为 $\lceil \log_k m \rceil$
改进：
 - 置换选择
 - 可使预处理阶段可以生成平均长度 $2p$ 的已排序片段，且 $O(\log p)$
 - 每个小片段进行堆排序，每次输出最小的值（即下标 0）
 - 如果下一个元素比输出值大，则放入下标 0，并对堆进行调整
 - 如果比输出值小，则无法放入当前的片段，写入内存尾部，留作下一个片段
- 多阶段归并：仅用 $k + 1$ 条磁带实现 k 路归并
- 最佳归并树：类似哈夫曼树

附：程序题标准写法

参数：

- 不会变化的，加 `const &`
- 需要反向影响的，加 `&`

指针操作：

- 新建 `node<elemType> *p = new node()`
- 复制 `node<elemType> *q = p`
- 置空 `p = NULL`
- 删除 `delete p`

数组操作：

- 新建 `elemType *a` 或 `elemType *a = new elemType[size]`
- 参数 `f(elemType a[])`
- 删除 `delete []a`

通用成员函数：`isEmpty`, `isFull`

数据结构：

- 线性表 seqList
- 链式表 linkList
- 栈
 - 线性存储 seqStack 链式存储 linkStack
 - 入栈 push 出栈 pop 读取栈顶元素 top
- 队列
 - 线性存储 seqQueue 链式存储 linkQueue
 - 入队 enqueue 出队 dequeue 读取队首元素 front